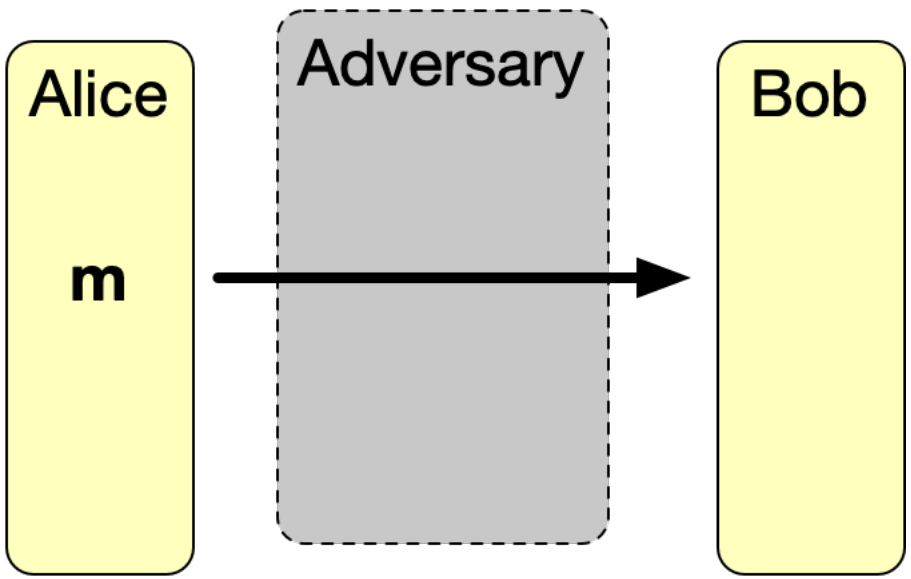
CS 3510 Algorithms: Cryptography and RSA

Aaron Hillegass
Georgia Tech

The Problem



One Time Pad Encryption

- Alice wants to send Bob a k-bit message m.
- Alice and Bob both have the same file of k random bits r.
- Encryption: Alice sends $m_e = m \oplus r$
- Decryption: Bob reads $m = m_e \oplus r$
- Why? $a \oplus b \oplus b = a$

One Time Pad Encryption is *totally secure*

However:

- Alice and Bob must have the same file of k random bits r.
- The adversary must not have that file.
- The random bits must never be used twice:

$$(m_1 \oplus r) \oplus (m_2 \oplus r) = m_1 \oplus m_2$$

(What is an easy way to increase the entropy of $m_1 \oplus m_2$?)

Symmetric Encryption (like AES)

- Alice and Bob both know a secret key k (think 128 or 256 bits).
- To send a message m, Alice encrypts it using k
- To decrypt the message, Bob decrypts it using k
- Really, really efficient (AES is often implemented in hardware)
- Salted with a random initialization vector so identical messages appear different.

Defining Secure

A system is *secure* if the best strategy available to the adversary is to repeatedly guess the key.

With a 128 bit key, there are 2^{128} possibilities.

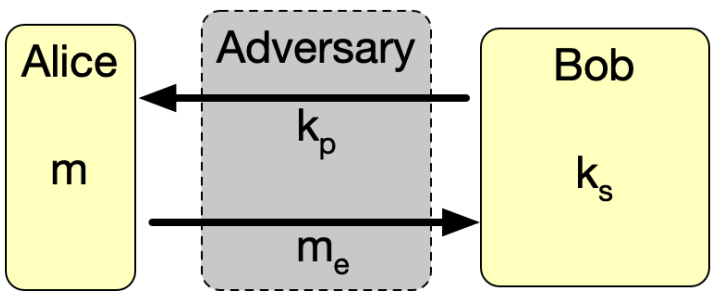
Is it breakable?

- How many milliseconds does a single CPU take to check a guess?
- How many CPUs do you have?
- How many years do you have/

But...securely sharing the secret key is really inconvenient.

Asymmetric (or “Public Key”) Encryption

- Bob has secret key k_s.
- He sends public key k_p to Alice.
- Alice encrypts m using k_p to m_e.
- Bob decrypts m_e it using k_s.
- Adversary can’t decrypt.
- Algorithms: Rivest–Shamir–Adleman algorithm (RSA) or elliptical curves



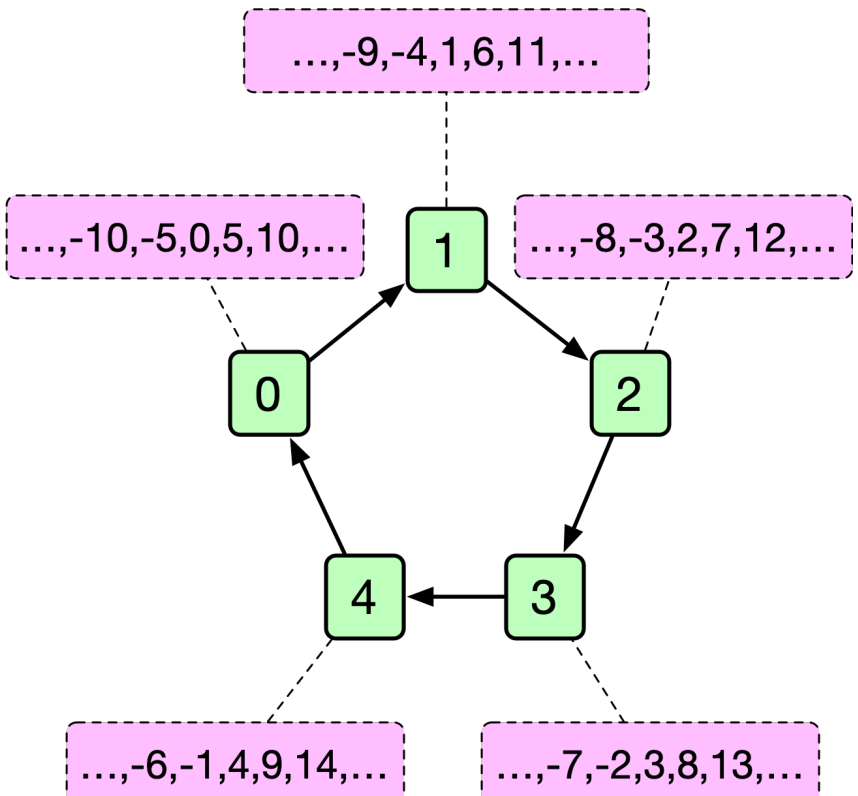
Doing public key encryption using OpenSSL


```
# Create a 4096-bit private key
openssl genrsa -out skey.pem 4096

# Extract a public key
openssl rsa -in skey.pem -out pubkey.pem -outform PEM -pubout

# Mail the public key to whoever will be decrypting your message

# Encrypt the message secret.txt using the private key
openssl pkeyutl -encrypt -inkey skey.pem -pubin -in m.txt -out m.enc
```

How does RSA work?

Ring of Integers Modulo n

- Denoted $\mathbb{Z}/n\mathbb{Z}$
- $a \equiv b (\mathrm{mod}\, n) \Leftrightarrow n$ divides $a - b$
- Members? $\{0, 1, ..., n-1\}$
- Addition is defined and has a zero
- Multiplication is defined and has a one
- Quiz: What is 3×4 in $\mathbb{Z}/5\mathbb{Z}$?



Exponents in $\mathbb{Z}/n\mathbb{Z}$

$x^y \bmod N$

```
1 define modexp(x, y, N):
2     if y == 0, return 1
3     z := modexp(x, y/2, N)
4     if y is even:
5         return z^2 mod N
6     else:
7         return x * z^2 mod N
```

$O((\log(N))^2 \log(y))$

Inverses in $\mathbb{Z}/n\mathbb{Z}$

Given $a \in \mathbb{Z}/n\mathbb{Z}$, define the inverse a^{-1} to be the member such that $aa^{-1} = 1$

Quiz: What is the inverse of 4 in $\mathbb{Z}/9\mathbb{Z}$?

What numbers have an inverse?

$a \neq 0$ has an inverse in $\mathbb{Z}/n\mathbb{Z}$ if and only if a and n are relatively prime.

That is $\gcd(a, n) = 1$.

Euclid's Algorithm for GCD

```
function gcd(a,b):
    if b == 0:
        return a
    else:
        return gcd(b, a mod b)
```

Correctness: if $d \mid a$ and $d \mid b$, then $d \mid a - b$

Termination: Assume (without loss of generality) that $a > b$.

Note that $0 \leq a \bmod b < b < a$; recursion will terminate.

Complexity of Euclid's Algorithm for GCD


```
function gcd(a,b):
    if b == 0:
        return a
    else:
        return gcd(b, a mod b)
```

- If a and b are n bits, mod operation is $O(n)$
- When $a > b$, $a \bmod b \leq \frac{a}{2}$. Recursion goes $O(n)$ deep.
- Complexity is $O(n^2)$
- Fast!

Fermat's Little Theorem

If p is prime and $0 < a < p$, then $a^{p-1} = 1 (\mathrm{mod}\, p)$

Proof:

Define set $S = \mathbb{Z}/p\mathbb{Z} - 0 = \{1, 2, ..., p-1\}$

Define set $aS = \{a1, a2, ..., a(p-1)\} \subset \mathbb{Z}/p\mathbb{Z}$

Want to show: $aS = S$.

1. $0 \notin aS$
2. Each element of aS is different

Proving: $0 \notin aS$

Suppose $0 \in aS$,

Then $\exists i$ such that $1 \leq i \leq p-1$ and $ai = 0$

p is prime, so $\exists a^{-1}$.

Then we note $i = a^{-1}ai = a^{-1}0 = 0$

Proving: Each element of aS is different

Suppose $\exists i, j \in S$ such that $i \neq j$ and $ai = aj$,

Then we note $i = a^{-1}ai = a^{-1}aj = j$

Shown: $aS = S$

Continuing Proof

Consider $\Pi S = \Pi aS$

$\Pi S = 1 \times 2 \times ... \times (p-1) \underset{p}{\equiv} (p-1)!$

$\Pi aS = 1a \times 2a \times ... \times (p-1)a \underset{p}{\equiv} (p-1)!a^{p-1}$

$(p-1)! \underset{p}{\equiv} (p-1)!a^{p-1}$

$(p-1)!$ is $1 \times 2 \times ... \times p-1$. Each term is non-zero. So each has an inverse. We can cancel them out.

$a^{p-1} \underset{p}{\equiv} 1$

Euler Phi

Define $\varphi(N)$ to be the number of numbers less than N that are relatively prime to N:

$$\varphi(N) = |\ \{a \mid 0 < a < N \text{ and } \gcd(a, N) = 1\}\ |$$

p is prime? $\varphi(p) = ?$

p and q are prime? $\varphi(pq) = ?$

Example: $\varphi(5 \times 7) = 4 \times 6$

There are 6 multiples of 5: 5, 10, 15, 20, 25, 30.

There are 4 multiples of 7: 7, 14, 21, 28.

There are 34 numbers less than 35. 34 - 6 - 4 = 24

$$\varphi(pq) = (pq - 1) - (p - 1) - (q - 1)$$

$$= pq - p - q + 1 = (p - 1)(q - 1)$$

p and q are prime? $\varphi(pq) = (p-1)(q-1)$

Euler's Theorem

A generalization of Fermat's Little Theorem:

If a and n are relatively prime, $a^{\varphi(n)} = 1 (\mathrm{mod}\, n)$

(FLT? n is prime, so $\varphi(n) = n - 1$)

Hardness Assumption

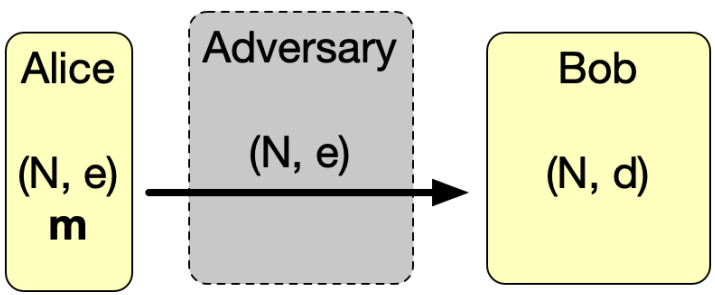
- If p and q are big primes, computing pq is easy.
- If N is the product of two big primes, factoring N into pq is hard.

Not proven! But we have never found a polynomial time algorithm.

(Except: Shor's Algorithm factors b-bit numbers in $O(b^3)$ on quantum computers.)

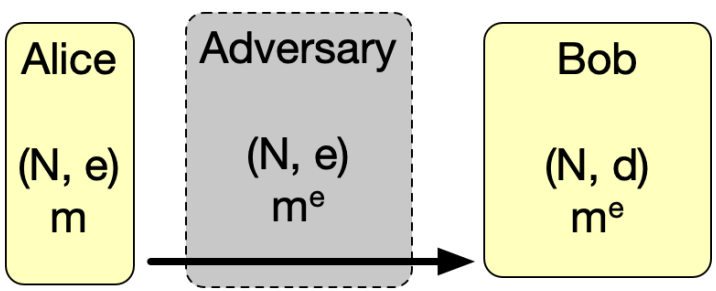
RSA

- Bob generates large primes p and q.
- Define $N = pq$.
- Bob calculates $\varphi(N) = (p-1)(q-1)$.
- Bob creates e, d where $e = d^{-1} (\mathrm{mod}\, \varphi(N))$.
- (Note e and $\varphi(N)$ must be relatively prime.)
- Public key (N, e) is sent to Alice.
- Bob keeps private key: (N, d).



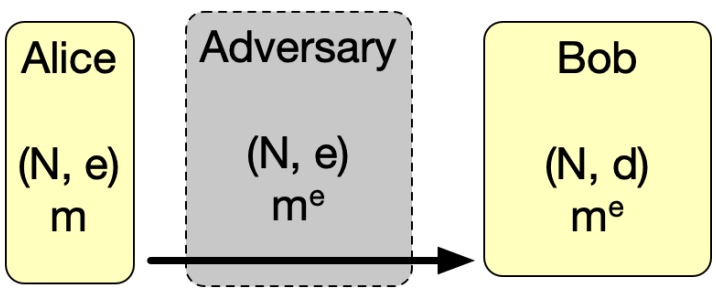
To send message

- Alice has message m. Computes $m^e \pmod N$, which is sent to Bob.
- Bob calculates $(m^e)^d \pmod N$ which is m.



The adversary?

- Has N, e, m^e.
- Needs d to read message.
- To find d, must compute $\varphi(N)$.
- To find $\varphi(N)$, must factor N into p and q – *which we think is very hard.*



Prove: $(m^e)^d = m(\mathrm{mod}\, N)$

We know: $ed = 1(\mathrm{mod}\, \varphi(N))$ so $\exists k$ such that $ed = k(\varphi(N)) + 1$

$(m^e)^d = m^{ed} = \left(m^k\right)^{\varphi(N)} m$

Note: m^k and N are almost certainly relatively prime

By Euler's Theorem: $\left(m^k\right)^{\varphi(N)} = 1(\mathrm{mod}\, N)$

$\left(m^k\right)^{\varphi(N)} m = 1m(\mathrm{mod}\, N)$

Voila! $(m^e)^d = m(\mathrm{mod}\, N)$

RSA in practice

RSA is really slow.

We encrypt the message with a random secret key using AES.

We encrypt just the secret key using RSA.

Finding multiplicative inverses

Remember: "Bob creates e, d where $e = d^{-1} \pmod{\varphi(N)}$"

- Pick a random candidate for e
- Relatively prime to $\varphi(N)$? If not repeat.
- Use Euler's Extended Algorithm to find d

Extended Euclidean Algorithm

For any integers, a and b, $\exists x, y \in \mathbb{Z}$ such that

$$ax + by = \gcd(a, b)$$

We can find x, y, and the gcd using the Extended Euclidean Algorithm. In our case:

$$ex + \varphi(N)y = 1$$

$$ex = 1 (\mathrm{mod}\, \varphi(N))$$

So $d = x$ in $\mathbb{Z}/\varphi(N)\mathbb{Z}$

Code


```
function extended_gcd(a, b)
    (old_r, r) := (a, b)
    (old_s, s) := (1, 0)
    (old_t, t) := (0, 1)

    while r ≠ 0 do
        quotient := old_r div r
        (old_r, r) := (r, old_r − quotient × r)
        (old_s, s) := (s, old_s − quotient × s)
        (old_t, t) := (t, old_t − quotient × t)

    output "x and y are", (old_s, old_t)
    output "greatest common divisor:", old_r
```

Implementation: Finding Big Primes

Generate a big random number, check to see if it is prime. If not, repeat.

How can you be pretty sure it is prime? Use Fermat's Little Theorem:

- Checking p? Come up with an a that is big, but smaller than p.
- Does $a^{p-1} = 1 (\mathrm{mod}\, p)$? p is probably prime.
- Repeat for several values of a

Carmichael Numbers: Numbers that are not prime but satisfy $a^{p-1} = 1 (\mathrm{mod}\, p)$ for all a relatively prime to p. Incredibly rare: 1 in 50 trillion.

Also: the Seive of Eratosthenes and Miller–Rabin.

Estimating π with randomness

To find π:

- Create billion (x, y) pairs where they are both between 0 and 1.
- Count how many are inside the unit circle. Multiply by 4. Divide by a billion.

Estimating π

Remember $\tan\left(\frac{\pi}{4}\right) = 1$:

- $\pi = 4\tan^{-1}(1)$

Approximate $\tan^{-1}(1)$ using a Taylor series.

- $\tan^{-1}(1) = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \frac{1}{9} - \frac{1}{11} + \ldots$
- Called “Leibniz’s formula for π”

Cryptographic Hash Functions

- A hash function takes any number of bits and outputs n bits (where n is usually 256 to 512 bits – too big to guess, small enough to store or transmit or use as a key in a dictionary.
- If you have the output, it is very, very difficult to come up with an input that generates it.
- Uses: passwords, digital signatures, file integrity, etc.
- MD5, SHA-1, SHA-2, SHA-3, BLAKE2,
- Slow hash functions: bcrypt

Zero Knowledge Proofs

The 3-coloring problem is NP-complete.

How could you prove that you had a 3-coloring without revealing the coloring itself?

All NP-Complete problems can be reduced to 3-COLOR.

Questions?

Slides by Aaron Hillegass

Based on lectures by Abrahim Ladha